工具性能说明（赛题交付件 #5）

测量轮次：§1–§5 于 2026-09-30丨§6 于 2026-10-17/18丨§7 于 2026-10-19 夜运行丨§8–§9 口径与复跑说明 文档基线：master `29df68f`（2026-10-22 同步）丨Python 3.12.1丨平台 x86_64（Windows 原生，无 WSL 依赖）

1. 自动化分析管线（autopilot v3，10 层，旗舰数据）

目标	源码规模	管线耗时	吞吐	lead 产出	管线峰值内存
dsoftbus 组件	1737 文件 / 681,529 行	**651.67s**	~1,046 行/s	**3033 条**（高危 580）	176.4MB
CJ FFI 63+ 仓	714 文件（含 489 CJ 命中）/ 173,901 行	**272.08s**	~639 行/s	**380 条**（高危 84）	58.4MB

dsoftbus 模式分布：P6_null_deref 2453 条；P8_use_after_free 511 条；P9_format_string 3 条；P8_double_free 6 条；P8_cross_uaf 60 条。 CJ FFI 模式分布：P6_null_deref 193 条；P8_use_after_free 40 条；P1_null_len 31 条；P8_double_free 2 条；P8_cross_uaf 26 条；P2_truncation 17 条；P3_nested_deref 10 条；P7_alloc_overflow 61 条。

> 口径说明：本轮为检测质量修复（跨函数 UAF 按 (callee,变量) 去重、判定收紧）后的**现役口径**—— 跨函数发现 60 条（dsoftbus），与历史未去重记录（889 条）不可直接比较；PoC 模板随 lead 生成。 内存为管线子进程自报峰值（Windows GetProcessMemoryInfo / POSIX getrusage）。 两处口径差异先说明，避免被读成矛盾：**651.67s** 是含子进程启停的墙钟，**652.08s**（§2） 是阶段自报，差 0.4s；**1,737** 是 `.c/.cpp` 文件数，§2 的 *文件 2967* 是 surface 统到的 全部文件（含头文件与其它扩展名），不是同一分母。 **"高危"的定义**：规则层 `severity=high` 的 lead（例如 P6 在路径上无判空即判 high、有判空则 medium；P4/P5 恒 high），由代码形态决定，**与置信度分数是两套口径**——两者都在 `findings.json` 里逐条可查，但不要把"高危条数"当成"置信度 ≥ N 的条数"。

2. 分阶段明细（同一轮实测）

阶段	耗时 (s)	Python峰值 (MB)	进程 RSS(MB)	产出
oh_dsoftbus_surface	—	—	—	文件 2967
oh_dsoftbus_autopilot	652.08	12.2	28.7	leads=3033 high=580 行数=681,529 管线内存=176.4MB
oh_dsoftbus_lexical	1.26	2.3	28.7	files=655, pattern_hits=1960
oh_dsoftbus_cjffi	13.72	0.0	28.7	note=files=1737 leads=25 -> cjffi_leads.json missing-both-checks: 16, files=1737, leads=25
oh_dsoftbus_snapshot	7.91	8.1	39.7	
cjffi_63repos_surface	—	—	—	文件 1178
cjffi_63repos_autopilot	272.27	1.5	39.7	leads=380 high=84 行数=173,901 管线内存=58.4MB
cjffi_63repos_lexical	0.42	0.1	39.7	files=2, pattern_hits=0
cjffi_63repos_cjffi	5.48	0.0	39.7	note=files=714 leads=37 -> cjffi_leads.json missing-both-checks: 32, files=714, leads=37

阶段	耗时 (s)	Python峰值 (MB)	进程 RSS(MB)	产出
cjffi_63repos_snapshot	10.48	10.1	44.2	

3. 总计与口径

- 本轮全阶段总耗时 **963.6s**（含 surface 统计、autopilot、词法、CJ 扫描、快照摘要）
- 管线峰值内存 **176.4MB**（autopilot 子进程自报；轻量阶段进程 RSS ≤ 44.2MB）
- 扫描覆盖率：autopilot 规则层覆盖树内**全部** .c/.cpp（无白名单），面数据如实记录于 findings.json 的 surface 字段
- 测量工具：`tools/perf_measure.py`（本轮起含 autopilot 阶段与 Windows 原生内存读取；修复了历史 rss 计算错误）

4. 环境与依赖说明

测量主机（§1–§7 同一台，无云端/容器）：

项	值
CPU	Intel Core i7-10875H @ 2.30GHz（8 核 16 线程）
内存	15.9 GB
操作系统	Windows 10 22H2（10.0.19045）
Python	3.12.1（测量时版本）
第三方依赖	无（`pip install -e . --no-deps`；扫描路径零运行时依赖）

- 扫描目标为真实 OpenHarmony 源码：dsoftbus 组件（681,529 行）+ CJ FFI 63 仓（136 个克隆仓中的 FFI 源面仓，173,901 行）
- 全部测量在 Windows 原生完成（零第三方依赖）；Clang AST / Joern 层需 WSL 环境单独测量
- 历史口径提示：早期记录的"742K 行 217s / 2466 lead"为检测质量修复前的测量，已被本节现役数据取代

5. Clang AST / Joern 层（WSL，选配）

引擎	目标	实测记录
Joern CPG（--joern 第 11 层）	dsoftbus	CPG 构建 158s + 查询 58s（独立运行记录）
Clang AST 引擎	按组件 profile	单文件 ~200-500ms，依赖编译数据库（profile 加载）

6. 增补实测（2026-10-17）

6.1 LLM 全量审查（glm-5.3 @ z.ai，26 组）

指标	实测
有效调用	46 次（含抖动重试轮），66,782 tokens（provider 上报口径）
单次时延（39 次有效草稿）	中位 12.7s / P90 17.3s / 最大 44.0s

指标	实测
传输抖动	当日两次独立失败（60s 超时、19s 传输错误，失败组随机）；分轮补跑 7 轮收满 26/26，终轮 verify completed
执行器	`tools/llm_review_run.py`（plan/run/verify，--timeout 透传）

组件批跑前置（同日）：securec+hilog 两作业 41 文件，clang-static-analyzer + clang-tidy（OH 预置 15.0.4，WSL 侧执行），产出 26 组 / 130 证据任务（`.cache/comp_review_20261017/`，全程约 3 分钟量级）。

6.2 P8 分支/闭包敏感化（dsoftbus 全树，同日）

指标	实测
旧语义（文本序 free→use）	448 条 lead
新语义（分支互斥 + 闭包过滤后）	**69 条**（移除 379，全部为裁定形态；2,383 个使用点带因抑制）
抑制开销	块区间分析 + 每使用点常数判定，扫描总时长与旧版同量级（文件 IO 主导）

6.3 LLM PoC 四道闸门（2026-10 上旬记录，10-21 起含帧归因）

端到端 ~30 秒/案（文本形态分类 + 链接集自推断编译 + 运行判定 + 真库帧归因），对照人工全流程约 2 小时（记录于导师路线交付，PR #132）。

闸门本身的开销口径：

环节	成本	说明
文本分类	毫秒级	纯正则状态机，不起进程
编译（含链接集推断）	随目标规模	上限 3 轮，每轮最多补 4 个源文件；`zig cc` 首次构建含 mingw CRT 重建（瞬态失败自动重试，不占模型预算）
ASan 可用性探测	每次验证一次，结果缓存	探不到即把静默腐蚀类的"没崩"降为 `undetermined`，不产生假 **refuted**
运行	单案 ≤10 s 超时	崩溃码判定，不解析 ASan 文本
帧归因（真库）	gdb 两轮，分钟级	musl 加载 + `llvm-addr2line -i`，按案触发而非批量

7. CVE 富集组件批量扫描（2026-10-19 夜运行，10-20 记录）

对 15 张 CVE 缺口工单的全部来源组件进行逐组件全管线扫描（autopilot v3 十层 + P6/P8 抑制采集）。运行窗口 2026-10-19 20:22:00 → 22:49:58（原始记录见 `docs/analysis/CVE_RICH_SCAN_20261020.md`）：

组件	行数	耗时(s)	吞吐(行/s)	lead	高危	P6 抑制
cJSON	19,950	41.5	481	226	7	127
bounds_checking_function	4,353	79.9	54	32	1	21
zlib	26,288	479.0	55	440	12	149
curl	247,337	1,169.2	212	5,352	84	2,005
openssl	564,273	955.0	591	10,793	98	4,870
ffmpeg	1,477,763	6,026.2	245	50,610	215	20,032
合计	**2,339,964**	**8,750.8**	**267**	**67,453**	**417**	**27,204**

- 总耗时 8,750.8s ≈ 2.43 小时（含 P6/P8 抑制采集双重扫描），逐组件串行执行；此前整目录扫 `third_party` 20 小时未完成，逐组件执行是可行性的前提
- 密度：高危 **1.78 条/万行**，lead **288 条/万行**
- **与作品报告的对应**：报告表 3-7 按整数秒登记（cJSON 42 s、合计 8,751 s），本表保留原始读数 41.5 s / 8,750.8 s——两者只差四舍五入，吞吐同为 267 行/s，不构成口径分歧
- **口径更正（2026-10-22）**：本节旧版写"417 / 2.34M = 17.8 条/万行，CVE 富集组件显著 高于一般代码"，是把 234 万行当成 23.4 万行算的，实际 1.78。更正后该密度低于 §1 的 dsoftbus（580 / 68.15 万 = 8.5 条/万行）与 CJ FFI（84 / 17.39 万 = 4.8 条/万行），说明"高危"是各轮自己的阈值口径（见 §1 注：规则层 severity，不是置信度），代码形态 分布又完全不同，跨批次不可比——故删除原结论，本表只作绝对量与吞吐报告
- **内存**：本轮只采墙钟，未逐组件记录进程峰值内存；同一台机器、同一条 autopilot 代码 路径的峰值内存见 §1（dsoftbus 176.4MB / CJ FFI 58.4MB），但组件规模更大时未复测，不据此外推本批次的内存占用

7.1 累计覆盖

扫描面	行数
dsoftbus（OH 核心组件）	681,529
CJ FFI 63 仓（136 克隆仓中）	173,901
CVE 富集 6 组件	2,339,964
合计	**3,195,394**

8. 说明

- P6 语义漏斗（2,662→13）为扫描产物上的排序/分层后处理，不构成独立扫描 阶段，不计入管线耗时口径；
- 6.1 的时延含网络往返，受当日通道质量影响，如实记录不作性能主张。

9. 复跑命令

```
# §1–§3：两棵树逐阶段测量（耗时 / 吞吐 / lead / 子进程自报峰值内存）
python tools/perf_measure.py

# 单棵（组件）树：计时与面数据在 findings.json 里
python -m ohvul.autopilot <tree> --output out\<name>
```

- 目标树在 `tools/perf_measure.py` 的 `main()` 顶部声明（`D:/cve_data/oh_dsoftbus`、`D:/cve_data/cjffi_sweep/repos`），换机器改这两行；
- §7 的逐组件行即对每个组件树单独跑一次 `ohvul.autopilot`，取 `findings.json` 的 `elapsed_seconds` 与 `surface`（文件数 / 行数估计）；
- 内存口径：`findings.json` 本身不记内存；`perf_measure.py` 让管线在子进程自报峰值（Windows `GetProcessMemoryInfo` / POSIX `getrusage`），与 §1 的 176.4MB 同源；
- 复跑前请确认机器空闲（本轮为单进程串行，未做并发压测，并发下的吞吐不作主张）。